

BACKGROUND: Review of Graphs & Graph Traversals

- ▶ General definitions; Representations;
- ▶ Terminology and more definitions;
- ▶ Graph Traversals; Depth-first search
- ▶ Topological sorting

Graphs – background

DEFINITION. A graph G is defined as a pair of sets $G = (V, E)$ with $E \subset V \times V$. So G represents a binary relation. The graph is **undirected** if the binary relation is symmetric. It is **directed** otherwise. V is the vertex set and E is the edge set.

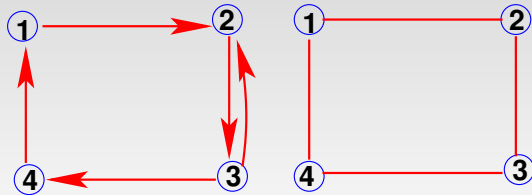
If R is a binary relation between elements in V then, we can represent it by a graph $G = (V, E)$ as follows:

$$(u, v) \in E \leftrightarrow u R v$$

Undirected graph $\leftrightarrow$ symmetric relation

Left: (1 R 2); (4 R 1); (2 R 3); (3 R 2); (3 R 4)

Right: (1 R 2); (2 R 3); (3 R 4); (4 R 1)



 1 Given the numbers 5, 3, 9, 15, 16, show the two graphs representing the relations

R1: Either $x < y$ or y divides x .

R2: x and y are congruent modulo 3. [$\text{mod}(x,3) = \text{mod}(y,3)$]

✦ $|E| \leq |V|^2$. For undirected graphs: $|E| \leq |V|(|V| + 1)/2$.

✦ A sparse graph is one for which $|E| \ll |V|^2$.

Basic Terminology & notation

- ▶ If $(u, v) \in E$, then v is **adjacent** to u . The edge (u, v) is **incident** to u and v .
- ▶ If the graph is directed, then (u, v) is an **outgoing** edge from u and **incoming** edge to v
- ▶ $Adj(i) = \{j | j \text{ adjacent to } i\}$
- ▶ The **degree** of a vertex v is the number of edges incident to v . Can also define the **indegree** and **outdegree**. (Sometimes self-edge $i \rightarrow i$ omitted)
- ▶ $|S|$ is the cardinality of set S [so $|Adj(i)| == \deg(i)$]
- ▶ A **subgraph** $G' = (V', E')$ of G is a graph with $V' \subset V$ and $E' \subset E$.

Representations of Graphs

- ▶ A graph is nothing but a collection of vertices (indices from 1 to n), each with a set of its adjacent vertices [in effect a 'sparse matrix without values']
- ▶ Therefore, can use any of the sparse matrix storage formats - omit the real values arrays.

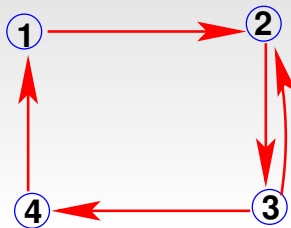
Adjacency matrix Assume $V = \{1, 2, \dots, n\}$. Then the *adjacency matrix* of $G = (V, E)$ is the $n \times n$ matrix, with entries:

$$a_{i,j} = \begin{cases} 1 & \text{if } (i,j) \in E \\ 0 & \text{Otherwise} \end{cases}$$

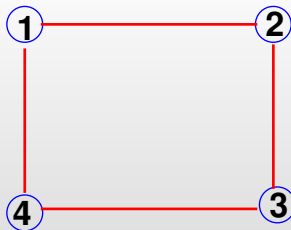
Representations of Graphs (cont.)

Example:

$$\begin{bmatrix} & 1 & & \\ & & 1 & \\ 1 & & & \\ & 1 & & 1 \end{bmatrix}$$



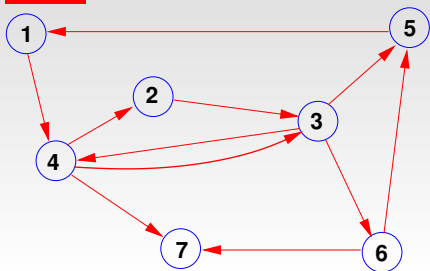
$$\begin{bmatrix} & 1 & & 1 \\ 1 & & & \\ & & 1 & \\ 1 & & & 1 \end{bmatrix}$$



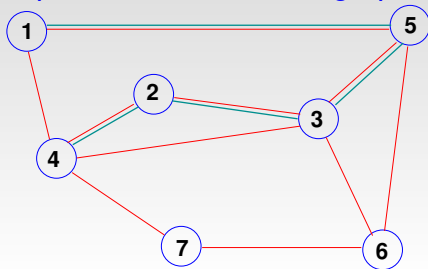
More terminology & notation

- ▶ For a given $Y \subset X$, the **section** graph of Y is the subgraph $G_Y = (Y, E(Y))$ where $E(Y) = \{(x, y) \in E \mid x \in Y, y \text{ in } Y\}$
- ▶ A section graph is a **clique** if all the nodes in the subgraph are pairwise adjacent ($\rightarrow$ dense block in matrix)
- ▶ A **path** is a sequence of vertices $w_0, w_1, \dots, w_k$ such that $(w_i, w_{i+1}) \in E$ for $i = 0, \dots, k - 1$.
- ▶ The **length** of the path $w_0, w_1, \dots, w_k$ is k (# of edges in the path)
- ▶ A **cycle** is a closed path, i.e., a path with $w_k = w_0$.
- ▶ A graph is **acyclic** if it has no cycles.

2 Find cycles in this graph:

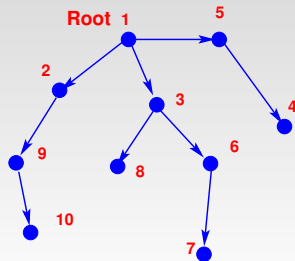
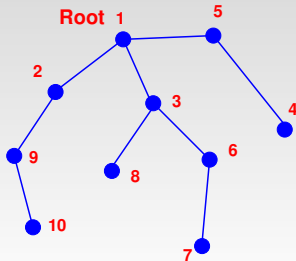


A path in an undirected graph



- ▶ A path $w_0, \dots, w_k$ is **simple** if the vertices $w_0, \dots, w_k$ are distinct (except that we may have $w_0 = w_k$ for cycles).
- ▶ An **undirected** graph is **connected** if there is path from every vertex to every other vertex.
- ▶ A **digraph** with the same property is said to be **strongly connected**

- ▶ The **undirected form** of a directed graph the undirected graph obtained by removing the directions of all the edges.
- ▶ Another term used “**symmetrized**” form -
- ▶ A directed graph whose undirected form is connected is said to be **weakly connected** or **connected**.
- ▶ **Tree** = a graph whose undirected form, i.e., symmetrized form, is acyclic & connected
- ▶ **Forest** = a collection of trees
- ▶ In a **rooted tree** one specific vertex is designated as a root.
- ▶ Root determines orientation of the tree edges in parent-child relation



- ▶ Parent-Child relation: immediate neighbors of root are children. Root is their parent. Recursively define children-parents
- ▶ In example: v_3 is parent of v_6, v_8 and v_6, v_8 are children of v_3 .
- ▶ Nodes that have no children are **leaves**. In example: v_{10}, v_7, v_8, v_4
- ▶ Descendent, ancestors, ...

Tree traversals

- ▶ Tree traversal is a process of visiting all vertices in a tree. Typically traversal starts at root.
- ▶ Want: systematic traversals of all nodes of tree – moving from a node to a child or parent
- ▶ **Preorder traversal:** Visit parent before children [recursively]
In example: $v_1, v_2, v_9, v_{10}, v_3, v_8, v_6, v_7, v_5, v_4$
- ▶ **Postorder traversal:** Visit children before parent [recursively]
In example : $v_{10}, v_9, v_2, v_8, v_7, v_6, v_3, v_4, v_5, v_1$

Graph Traversals – Depth First Search

- ▶ Issue: systematic way of visiting all nodes of a **general** graph
- ▶ Two basic methods: Breadth First Search (will see later) & ...

◆ Depth-First Search

Algorithm $List = DFS(G, v)$ (DFS from v)

- ▶ Visit and Mark v ;
- ▶ for all edges (v, w) do
 - if w is not marked then $List = DFS(G, w)$
 - Add v to top of list produced above

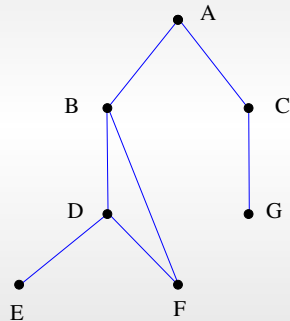
- ▶ If G is undirected and connected, all nodes will be visited
- ▶ If G is directed and strongly connected, all nodes will be visited

Depth First Search – undirected graph example

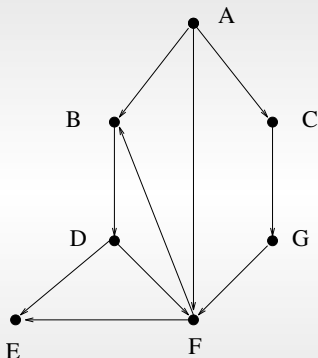
✦ We now use column-oriented graph representation: $j \rightarrow i$ if $a_{ij} \neq 0$

✦ Assume: adjacent nodes listed alphabetically

 3 DFS traversal from A ?



Depth First Search – directed graph example



✦ Assume: Adjacent nodes in alphabetical order

 4 DFS traversal from A?

```

function [Lst, Mark] = dfs(u, A, Lst, Mark)
%% function [Lst, Mark] = dfs(u, A, Lst, Mark)
%% dfs from node u -- Recursive
%%-----
[ii, jj, rr] = find(A(:,u));
Mark(u) = 1;
for k=1:length(ii)
    v = ii(k);
    if (~Mark(v))
        [Lst, Mark] = dfs(v, A, Lst, Mark);
    end
end
Lst = [u,Lst]

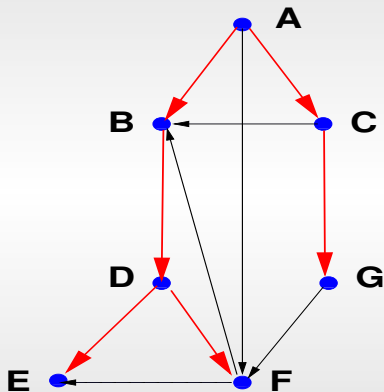
```

Depth-First-Search Tree: Consider the parent-child relation: v is a parent of u if u was visited from v in the depth first search algorithm. The (directed) graph resulting from this binary relation is a tree called the Depth-First-Search Tree. To describe tree: only need the parents list.

✦ To traverse all the graph we need a $\text{DFS}(v, G)$ from each node v that has not been visited yet – so add another loop. Refer to this as:

$\text{DFS}(G)$: When a new vertex is visited in DFS, some work is done. Example: we can build a stack of nodes visited to show order (reverse order: easier) in which the node is visited.

Back edges, forward edges, and cross edges



- ✦ Thick red lines: DFS traversal tree from A
- ✦ $A \rightarrow F$ is a Forward edge
- ✦ $F \rightarrow B$ is a Back edge
- ✦ $C \rightarrow B$ and $G \rightarrow F$ are Cross-edges.

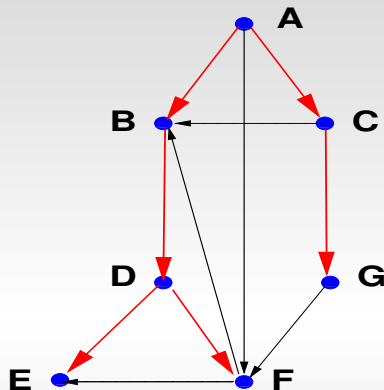
- ✦ Consider 'List' produced by DFS:

$Lst=[A, C, G, B, D, F, E]$

- ✦ Order in list is important for some algorithms
- ✦ Notice: Label nodes in List from 1 to n

- ✦ Then:

- ▶ Tree-edges / Forward edges : labels increase in $\rightarrow$
- ▶ Cross edges : labels in/de-crease in $\rightarrow$ [depends on labeling]
- ▶ Back-edges : labels decrease in $\rightarrow$



Properties of Depth First Search

- ✦ If G is a connected undirected (or strongly connected) graph, then each vertex will be visited once and each edge will be inspected at least once.
- ✦ Therefore, for a connected undirected graph, The cost of DFS is $O(|V| + |E|)$
- ✦ If the graph is undirected, then there are no cross-edges. (all non-tree edges are called 'back-edges')

Theorem: A directed graph is acyclic iff a DFS search of G yields no back-edges.

- ✦ Terminology: Directed Acyclic Graph or **DAG**

The problem:

Given a **Directed Acyclic Graph** (DAG), order the vertices from **1** to **n** such that, if (u, v) is an edge, then u appears before v in the ordering.

- ✦ Equivalently, label vertices from 1 to n so that in any (directed) path from a node labelled k , all vertices in the path have labels $> k$.
 - ▶ Numerous Applications: Prerequisite requirements in a program; Scheduling of tasks for any project; Parallel tasks in algorithms; etc.

Topological Sorting: A first algorithm

Property exploited: An acyclic Digraph has at least one vertex with indegree 0.



5

Prove this

Algorithm:

- 1 Find nodes with in-deg == 0. Label them as $1, 2, \dots, k$;
- 2 Remove these vertices and all edges incident from them
- 3 Resulting graph is again acyclic $\rightarrow \exists$ nodes with in-deg = 0. Label these nodes as $k + 1, k + 2, \dots$. Remove them and adj. vertices.
- 4 Repeat..



6

Explore implementation aspects.

Alternative method: Topological sort from DFS

- ▶ Depth first search traversal of graph
- ▶ ‘post-order traversal’ of the DFS tree.

Algorithm $Lst = Tsort(G)$

```
Mark = zeros(n,1); Lst =  $\emptyset$ 
for v=1:n do
    if (Mark(v)== 0)
        [Lst, Mark] = dfs(v, G, Lst, Mark);
    end
end
```

✦ $dfs(v, G, Lst, Mark)$ is the DFS(G, v) which adds v to the top of Lst after finishing the traversal from v

$Lst = DFS(G, v)$

- ▶ Visit and Mark v ;
- ▶ for all edges (v, w) do
 - if w is not marked then $Lst = DFS(G, w)$
- ▶ $Lst = [v, Lst]$

✦ Topological order given by the final Lst array of Tsort

 7 Explore implementation issue

 8 Implement in matlab

 9 Show correctness [i.e.: is this indeed a topol. order? hint: no back-edges in a DAG]